

JSON WEB TOKEN

PROTOCOLES D'AUTHENTIFICATION

LES LIMITES DE LA SESSION

Nous avons vu que le protocole HTTP est amnésique (**stateless**). Pour lui donner une mémoire, nous avons utilisé des **sessions** stockées dans la **RAM** du serveur.

Cette **architecture statefull** pose néanmoins plusieurs **problèmes** :

- Si 100 000 utilisateurs se connectent, le serveur doit stocker 100 000 objets, ce qui consomme énormément de mémoire vive ;
- Si le serveur redémarre ou crash, la RAM se vide et tous les utilisateurs sont déconnectés ;
- Si l'application grandit et qu'on utilise 3 serveurs différents (Load Balancing *1), la session créée sur le serveur A sera inconnue du serveur B (**scalabilité**).

La solution moderne consiste à rendre le serveur à nouveau amnésique (**stateless**) mais intelligent. C'est le rôle du **JWT** !



CRYPTOGRAPHIE

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

LES NOTIONS CRYPTOGRAPHIQUES

Pour l'authentification, nous manipulons ces **quatre notions cryptographiques** qui ont des rôles différents :

1. L'encodage (ex: Base64)

👉 Transforme un format de donnée en un autre pour qu'il puisse être transporté facilement.

⚠️ N'importe qui peut le décoder sans clé. **Il ne suit pas un procédé cryptographique !**

2. Le hachage (ex: Bcrypt)

👉 Transforme une donnée en une chaîne de caractères de taille fixe. **Irréversible** => on ne peut pas recréer le mot de passe à partir du hash.

✅ Sécurité élevée. Idéal pour stocker des **secrets**.

3. Le chiffrement (ex: HTTPS)

👉 Rend une donnée illisible pour quiconque ne possède pas la **clé secrète de déchiffrement**. C'est une opération **réversible** (si on a la clé).

✅ Sécurité très élevée.

4. La signature numérique (JWT)

👉 On prend une donnée publique, on y applique un calcul mathématique avec une clé secrète pour générer une empreinte (un "sceau").

✅ Sécurité très élevée. Elle ne cache pas la donnée mais prouve mathématiquement qu'elle **provient du serveur** et qu'elle **n'a pas été modifiée**.

LES NOTIONS CRYPTOGRAPHIQUES

Concept	Action principale	Réversible ?	Nécessite une clé ?	Cas d'usage
Encodage	Traduire	Oui (facilement)	Non	Envoi du header/payload (JWT)
Hachage	Détruire	Non (à sens unique)	Non	Stockage des mots de passe en base
Chiffrement	Cacher	Oui (si on a la clé)	Oui	Transport réseau (HTTPS)
Signature	Authentifier	Non applicable	Oui	Vérification du jeton (JWT)



JWT

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

QU'EST-CE QU'UN JWT ?

Un **JWT** (pour JSON Web Token) est un standard ([RFC 7519](#)) qui définit un moyen **compact** et **autonome** pour transmettre des informations de manière sécurisée entre deux machines.

Contrairement au **Session Id** qui n'est qu'une clé pour la session, le JWT **contient directement** les données de l'utilisateur (id, email, rôle, etc.).

Le serveur n'a plus besoin d'interroger sa RAM. Il lui suffit de lire le contenu du jeton (ou *token*).

Les informations ne sont pas cachées mais **signées mathématiquement**.

Ici, la **cryptographie** est utilisée pour lier les données à une **clé secrète** connue du serveur uniquement pour garantir **l'intégrité** du jeton (impossible à modifier sans briser la signature).

LA STRUCTURE D'UN JWT ?

Un JWT est une **chaîne de caractères** divisée en **3 parties** séparées par des points **.** :



xxxxxx . **yyyyyy** . **zzzzzz** :

1. Le header (ou en-tête) : **xxxxxx**

- Décrit le **type de jeton** *2 et l'**algorithme de signature** *3 utilisé ;

2. Le payload (ou charge utile) : **yyyyyy**

- Contient les "Claims" (ou *revendications*) => les données que l'on veut transmettre (id, email, rôle, etc.) ;

3. La signature : **zzzzzz**

- C'est le sceau de garantie généré en combinant le **header**, le **payload** ET un **secret** que **seul le serveur connaît** (protégé dans le fichier **.env**).

LE HEADER DÉCODÉ

Le **header** décodé est du **JSON** :

A code editor window with a dark blue background and a light blue border. The title bar at the top says "header JWT". Inside the editor, there is a JSON object with two properties: "alg" and "typ". The "alg" property has the value "HS256" and a comment "// L'algorithme de signature". The "typ" property has the value "JWT" and a comment "// Le type de document".

```
{  
  "alg": "HS256", // L'algorithme de signature  
  "typ": "JWT"    // Le type de document  
}
```

- **"typ"** (type) → Dit au serveur : « Je suis un jeton au format JWT » ;
- **"alg"** (algorithme) → Dit au serveur : « Pour vérifier mon sceau de sécurité, applique la formule HS256 avec ta clé secrète ».

À QUOI RESSEMBLE UN JWT ?

`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjEwNjYyOTQxMn0.KMUFsIDTnFmyG3nMiGM6H9FNfUR0f3wh7SmqJp-QV30`

Le **header** et le **payload** sont encodés en Base64Url *4. N'importe qui interceptant le jeton peut lire son contenu !

⚠ On ne place donc **JAMAIS** d'informations sensibles dans le **payload** d'un JWT !

OÙ STOCKER LE JETON ?

Une fois généré par le serveur, le JWT doit être stocké par le client et renvoyé à chaque requête. Deux écoles existent :

1. Dans le `localStorage`

- Le frontend stocke le JWT dans le `localStorage` *5 et l'envoie dans l'en-tête HTTP `Authorization: Bearer <token>`;
- ⚠ Si un pirate injecte un script JavaScript malveillant sur votre site, il peut lire le `localStorage` et voler le jeton.

2. Dans un cookie sécurisé (recommandé)

- Le serveur dépose le jeton dans le navigateur dans l'en-tête HTTP `Set-Cookie`. Le navigateur le stocke et le joint à chaque future requête HTTP ;
- 🔒 En activant le flag `HttpOnly`, le navigateur interdit au code JS de lire le cookie.

LE STANDARD « BEARER TOKEN »

Dans certaines applications est utilisé un en-tête HTTP `Authorization Bearer` et non un cookie.

Le jeton est stocké par le client (ex. application mobile) et injecté manuellement dans chaque requête :



```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Critère	Approche « cookie »	Approche « en-tête »
Stockage Client	Géré par le navigateur	localStorage ou mémoire app
Envoi au serveur	Automatique par le navigateur	Manuel via le code Frontend
Cible idéale	Applications Web classiques / SPA	Applications Mobiles / API Publiques
Sécurité XSS	 Totalement protégé (HttpOnly)	 Vulnérable (Vol de token possible)

 Le format *Bearer* est le standard universel pour les architectures mobiles et multi-plateformes. Le format *cookie* reste néanmoins le roi de la sécurité pour le web standard.

QUESTION OUVERTE

Puisque le payload d'un JWT est juste encodé en Base64 et que n'importe qui peut le décoder et le lire...

Que se passe-t-il si un utilisateur mal intentionné modifie son rôle "USER" par "ADMIN" dans le payload avant de le renvoyer au serveur ?



IMPLÉMENTATION

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

ARCHITECTURE

Pour l'architecture, nous allons utiliser une structure **modulaire propre** (avec des routeurs Express dédiés) :

```
Architecture

/demo
├── /config
│   └── db.js           # Initialisation SQLite
├── /middlewares
│   └── authCheck.js    # Vigile de vérification du JWT
├── /routes
│   └── auth.js         # Routes Login, Me et Logout
├── /views
│   └── dashboard.html  # Page protégée HTML
├── /public
│   ├── login.html     # Page publique de connexion
│   ├── login.js       # Script frontend pour le login
│   └── dashboard.js    # Script frontend pour le dashboard
├── .env               # Secret d'infrastructure
├── package.json
└── server.js          # Point d'entrée léger
```

DÉPENDANCES ET CONFIGURATION

Si ça n'est pas déjà fait, on installe les modules nécessaires. Les seules nouveautés sont les librairies `jsonwebtoken` et `dotenv` *6.

👉 `npm i express better-sqlite3 bcrypt jsonwebtoken dotenv`

Ainsi, dans le fichier de configuration de l'environnement `.env`, on définit le **secret** qui servira à **signer numériquement** nos JWT :

👉 `node -e`

`"console.log(require('crypto').randomBytes(32).toString('hex'))"`

PORT=3000

JWT_SECRET=7fd9c4ccc1ef7332aaef69cf0fb4e5395a72f8ea51af285e090fc1be53e8af98

[Copier le code](#)

INITIALISATION DE LA BDD

```
config/db.js

const Database = require('better-sqlite3');
const bcrypt = require('bcrypt');

const db = new Database('jwt_demo.db');

db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT UNIQUE,
    password_hash TEXT,
    role TEXT
  )
`);

// Création d'un compte de test si inexistant
const user = db.prepare('SELECT * FROM users WHERE email = ?').get('chris@chevalier.com');
if (!user) {
  const hash = bcrypt.hashSync('Database2026!', 10);
  db.prepare('INSERT INTO users (email, password_hash, role) VALUES (?, ?, ?)')
    .run('chris@chevalier.com', hash, 'ADMIN');
}

module.exports = db;
```

[Code à copier](#)

```

routes/auth.js

const express = require('express');
const router = express.Router();
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const db = require('../config/db');

router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);
    if (!user || !(await bcrypt.compare(password, user.password_hash))) {
      return res.status(401).json({ erreur: 'Identifiants incorrects' });
    }

    const payload = {
      id: user.id,
      email: user.email,
      role: user.role
    };

    const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '15m' });

    res.cookie('token', token, {
      httpOnly: true, // Anti-XSS
      secure: false, // Mettre à true en production (HTTPS)
      sameSite: 'strict', // Anti-CSRF
      maxAge: 900000 // 15 minutes en millisecondes
    });

    res.json({ message: 'Connexion réussie.' });

  } catch (error) {
    res.status(500).json({ erreur: 'Erreur serveur' });
  }
});

router.post('/logout', (req, res) => {
  res.clearCookie('token');
  res.json({ message: 'Déconnexion réussie.' });
});

module.exports = router;

```

[Copier le code](#)

EXPLICATIONS

Le fichier `/routes/auth.js` orchestre le **cycle de vie du jeton** côté serveur. Voici les mécanismes essentiels à retenir :

- **La trilogie du JWT**

1. **Le payload** : On y embarque le strict minimum nécessaire à l'application (id, email, role) au format JSON ;
2. **La signature** : La méthode `jwt.sign()` verrouille cryptographiquement ces données en utilisant le `JWT_SECRET` fourni par `dotenv` ;
3. **L'expiration** : L'option `expiresIn: '15m'` donne une durée de vie très courte au jeton. C'est notre parade principale puisqu'un JWT ne peut pas être révoqué à distance avant sa mort naturelle.

- **Le transport**

Le JWT est envoyé dans un **cookie sécurisé**.

- **La déconnexion**

Le serveur étant amnésique, il n'a aucun registre en RAM à nettoyer. Pour déconnecter l'utilisateur, il utilise `res.clearCookie('token')` qui ordonne au navigateur de **détruire le fichier cookie** contenant le jeton.

```
const jwt = require('jsonwebtoken');

function checkJWT(req, res, next) {
  const cookieHeader = req.headers.cookie;
  if (!cookieHeader) {
    return res.status(401).json({ erreur: 'Accès refusé. Aucun jeton fourni.' });
  }

  const tokenCookie = cookieHeader.split(';').find(c => c.trim().startsWith('token='));
  if (!tokenCookie) {
    return res.status(401).json({ erreur: 'Accès refusé. Jeton introuvable.' });
  }

  const token = tokenCookie.split('=')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({ erreur: 'Jeton invalide ou expiré.' });
  }
}

module.exports = checkJWT;
```

EXPLICATIONS

`/middlewares/authCheck.js` intercepte les requêtes vers les **routes protégées** pour valider le jeton en **3 étapes** :

1. L'extraction manuelle

Le code traite directement la chaîne de caractères de l'en-tête HTTP pour isoler le cookie `token` (`req.headers.cookie`) ;

2. Le contrôle de sécurité

La méthode `jwt.verify()` **décode** le jeton ET vérifie sa **signature numérique** à l'aide du `JWT_SECRET`. Si le jeton a été falsifié par l'utilisateur ou si sa date de fin est dépassée, le bloc `catch` lève une erreur `401` ;

3. La transmission des droits

Une fois la signature validée, les données du payload (id, email, role) sont injectées dans l'objet de la requête `req`. Elles deviennent ainsi disponibles pour les routes suivantes.

💡 Contrairement au mécanisme des sessions, le serveur n'interroge plus sa RAM pour savoir qui est connecté. Il valide mathématiquement les données fournies par le client.

PAGE DE CONNEXION (HTML/JS)

public/login.html

```
<!DOCTYPE html>
<html lang="fr-FR">
<head>
  <meta charset="UTF-8">
  <title>Connexion JWT</title>
</head>
<body>
  <h2>Connexion</h2>
  <div id="error" style="color: red; display: none;"></div>
  <form id="login-form">
    <input type="email" id="email" placeholder="Email" required><br><br>
    <input type="password" id="password" placeholder="Mot de passe" required><br><br>
    <button type="submit">Se connecter</button>
  </form>
  <script src="login.js"></script>
</body>
</html>
```

[Code à copier](#)

public/login.js

```
document.getElementById('login-form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const email = document.getElementById('email').value;
  const password = document.getElementById('password').value;
  const errorDiv = document.getElementById('error');

  const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  });

  if (response.ok) {
    // Le navigateur a reçu le cookie JWT, on redirige vers le dashboard sécurisé
    window.location.href = '/dashboard';
  } else {
    const data = await response.json();
    errorDiv.innerText = data.erreur;
    errorDiv.style.display = 'block';
  }
});
```

[Code à copier](#)

PAGE PROTÉGÉE

views/dashboard.html

```
<!DOCTYPE html>
<html lang="fr-FR">
<head>
  <meta charset="UTF-8">
  <title>Administration - Tableau de bord</title>
</head>
<body>
  <h1 style="color: green;">Accès autorisé au tableau de bord</h1>
  <p>Bienvenue, <strong id="user-email">...</strong></p>
  <p>Votre rôle : <span id="user-role">...</span></p>

  <button id="logout-btn">Déconnexion</button>

  <script src="./dashboard.js"></script>
</body>
</html>
```

[Code à copier](#)

public/dashboard.js

```
// Récupération des infos de l'utilisateur connecté via l'API protégée
fetch('/api/user/me')
  .then(res => {
    if(!res.ok) window.location.href = '/';
    return res.json();
  })
  .then(data => {
    document.getElementById('user-email').innerText = data.email;
    document.getElementById('user-role').innerText = data.role;
  });

// Déconnexion
document.getElementById('logout-btn').addEventListener('click', async () => {
  await fetch('/api/auth/logout', { method: 'POST' });
  window.location.href = '/';
});
```

[Code à copier](#)

POINT D'ENTRÉE DE L'ARCHITECTURE

```
server.js

require('dotenv').config();
const express = require('express');
const path = require('path');
const authRoutes = require('./routes/auth');
const checkJWT = require('./middlewares/authCheck');

const app = express();

app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.use('/api/auth', authRoutes);

app.get('/api/user/me', checkJWT, (req, res) => {
  res.json({ email: req.user.email, role: req.user.role });
});

app.get('/dashboard', checkJWT, (req, res) => {
  res.sendFile(path.join(__dirname, 'views', 'dashboard.html'));
});

app.get(/.*$/, (req, res) => {
  res.redirect('/login.html')
})

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Serveur lancé => http://localhost:${PORT}`));
```

[Code à copier](#)

EXPLICATIONS

Le fichier principal `server.js` assemble les briques de l'**architecture stateless** de manière légère :

- `app.use('/api/auth', authRoutes)` centralise la logique de connexion et de déconnexion.
- La route `GET /api/user/me` consomme l'objet `req.user` (id, email, role). 💡 Cet objet a été extrait et validé par notre middleware de sécurité `checkJWT()`.
- L'accès physique au fichier HTML (`/dashboard`) est lui aussi verrouillé par le middleware `checkJWT()`. Si le jeton est absent ou invalide, le navigateur est bloqué.
- Le middleware de redirection automatique `app.get(/.*$/, ...)` renvoie tout utilisateur non authentifié ou perdu vers la page `login.html`.



RENOUVELLEMENT DE JETON

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

EXPIRATION ET RÉVOCATION

Problèmes

1. Un JWT est valide jusqu'à sa date d'expiration. S'il expire au bout de 15 minutes, l'expérience utilisateur est dégradée ;
2. Le serveur ne peut pas, non plus, le « détruire » à distance.

Solutions

1. Donner une durée de vie courte et maîtrisée au jeton (entre 15 minutes et 1 heure) ;
2. Créer un « jeton de rafraichissement » (stocké en base pour 7 jours, par exemple) permettant de régénérer de nouveaux JWT si toujours valide.

RENOUVELLEMENT DE JETON

Pour ne pas déconnecter l'utilisateur toutes les 15 minutes, on utilise donc un *refreshToken*.



Voici le flux logique :

1. **À la connexion**, le serveur envoie **2 cookies** :
 1. Le *token* (jeton d'accès valide pour 15 minutes ou moins) ;
 2. Le *refreshToken* (opaque, stocké en base et valide pour 7 jours).
2. **À l'expiration du jeton d'accès**, le frontend reçoit une erreur **401**. Il appelle la route `/refresh` ;
3. Le serveur vérifie le *refreshToken*. S'il est valide, il renvoie un **nouveau jeton d'accès**.

AJOUTE DE LA TABLE *REFRESH_TOKENS*

```
config/db.js

// ...
db.exec(`
  CREATE TABLE IF NOT EXISTS refresh_tokens (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    token TEXT UNIQUE NOT NULL,
    user_id INTEGER NOT NULL,
    expires_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
  );
`);
// ...
```

[Code à copier](#)

CONNEXION

```
routes/auth.js

// ...
const crypto = require('crypto');

router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);
    if (!user || !(await bcrypt.compare(password, user.password_hash))) {
      return res.status(401).json({ erreur: 'Identifiants incorrects' });
    }

    const token = jwt.sign(
      { id: user.id, email: user.email, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '15s' }
    );

    const refreshToken = crypto.randomBytes(40).toString('hex');
    const expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString();

    db.prepare('INSERT INTO refresh_tokens (token, user_id, expires_at) VALUES (?, ?, ?)')
      .run(refreshToken, user.id, expiresAt);

    res.cookie('token', token, { httpOnly: true, sameSite: 'strict', maxAge: 15000 });
    res.cookie('refreshToken', refreshToken, { httpOnly: true, sameSite: 'strict', maxAge: 7 *
24 * 60 * 60 * 1000 });

    res.json({ message: 'Connexion réussie.' });
  } catch (error) {
    res.status(500).json({ erreur: 'Erreur serveur' });
  }
});

// ...
```

[Code à copier](#)

EXPLICATIONS

Le jeton d'accès (ou *accessToken*) est autonome et éphémère (`expiresIn: '15s'`). S'il est intercepté, sa durée de vie limite l'impact du vol.

⚠ Généralement, la durée de vie d'un *accessToken* en production se situe plutôt **entre 15 minutes et 1 heure** mais nous le réduisons ici à 15 secondes dans un but purement pédagogique.

Le jeton de rafraîchissement (ou *refreshToken*) est une chaîne opaque générée par le serveur (une chaîne aléatoire de 80 caractères hexadécimaux). Contrairement au jeton d'accès, il ne contient aucune donnée utilisateur en clair.

Le navigateur reçoit deux cookies distincts. L'*accessToken* expire physiquement au bout de 15 secondes tandis que le *refreshToken* reste valide pendant 7 jours (en ms et transformé au format universel `toISOString()` pour la stocker en base).

L'enregistrement du *refreshToken* en base réintroduit un état de contrôle minimal. C'est le composant qui permettra au serveur d'invalider ou de révoquer une connexion à distance avant son terme naturel.

RENOUVELLEMENT

```
routes/auth.js

// ...
router.post('/refresh', (req, res) => {
  const refreshToken = req.headers.cookie?.split(';').find(c =>
c.trim().startsWith('refreshToken='))?.split('=')[1];
  if (!refreshToken) return res.status(401).json({ erreur: 'Accès refusé' });

  const storedToken = db.prepare('SELECT * FROM refresh_tokens WHERE token =
?').get(refreshToken);

  if (!storedToken || new Date() > new Date(storedToken?.expires_at)) {
    return res.status(401).json({ erreur: 'Session expirée, reconnectez-vous' });
  }

  const user = db.prepare('SELECT * FROM users WHERE id = ?').get(storedToken.user_id);
  const newToken = jwt.sign(
    { id: user.id, email: user.email, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '15s' }
  );

  res.cookie('token', newToken, { httpOnly: true, sameSite: 'strict', maxAge: 15000 });
  res.json({ message: 'Jeton d\'accès rafraîchi.' });
});
// ...
```

[Code à copier](#)

EXPLICATIONS

Cette route orchestre le prolongement de la connexion sans imposer une reconnexion manuelle à l'utilisateur :

- L'API extrait le *refreshToken* opaque depuis les en-têtes de la requête. S'il est manquant, l'accès est immédiatement verrouillé par un code 401.
- Une requête cible la base pour vérifier l'existence du jeton. Deux barrières de sécurité sont appliquées :
 - Le jeton doit être enregistré dans la table `refresh_tokens` ;
 - La date actuelle ne doit pas avoir dépassé la date limite `expires_at` (sur 7 jours).
- Si les validations réussissent, le serveur identifie l'utilisateur associé et génère un **nouveau jeton d'accès** d'une nouvelle validité de 15 secondes.
- Le nouveau jeton d'accès est renvoyé au navigateur dans le cookie `token`.

DÉCONNEXION

```
routes/auth.js

// ...
router.post('/logout', (req, res) => {
  const refreshToken = req.headers.cookie?.split(';').find(c =>
c.trim().startsWith('refreshToken='))?.split('=')[1];

  if (refreshToken) {
    db.prepare('DELETE FROM refresh_tokens WHERE token = ?').run(refreshToken);
  }

  res.clearCookie('token');
  res.clearCookie('refreshToken');
  res.json({ message: 'Déconnexion et révocation réussies.' });
});
// ...
```

[Code à copier](#)

EXPLICATIONS

La déconnexion ne doit pas simplement être esthétique côté client ; elle doit garantir l'invalidation absolue de la connexion utilisateur :

1. **Le principe de révocation** (côté serveur) : Le serveur exécute une requête `DELETE` pour détruire la **ligne SQLite** correspondant au *refreshToken*. Ainsi, même si un pirate possède encore ce jeton physique, il devient inutile car absent de la base ;
2. **Le nettoyage client** (cookies) : Le serveur ordonne au navigateur d'effacer les cookies `token` et `refreshToken` via l'instruction `res.clearCookie()`.

```
public/dashboard.js

async function fetchWithRetry(url, options = {}) {
  let response = await fetch(url, options);

  if (response.status === 401) {
    console.warn("JWT expiré. Tentative de rafraîchissement automatique...");
    const refreshResponse = await fetch('/api/auth/refresh', { method: 'POST' });

    if (refreshResponse.ok) {
      console.log("Nouveau JWT obtenu ! Rejeu de la requête initiale.");
      response = await fetch(url, options);
    } else {
      console.error("Le Refresh Token est également invalide ou révoqué.");
      window.location.href = '/login.html';
      return null;
    }
  }
  return response;
}

fetchWithRetry('/api/user/me')
  .then(res => {
    if (res) return res.json();
  })
  .then(data => {
    if (data) {
      document.getElementById('user-email').innerText = data.email;
      document.getElementById('user-role').innerText = data.role;
    }
  })
  .catch(err => console.error("Erreur d'affichage :", err));

document.getElementById('logout-btn').addEventListener('click', async () => {
  await fetch('/api/auth/logout', { method: 'POST' });
  window.location.href = '/login.html';
});
```

[Code à copier](#)

EXPLICATIONS

Le script client orchestre la résilience de la connexion grâce au mécanisme de rejeu ("Retry Pattern") :

- Au lieu d'appeler le fetch JS, le frontend utilise une fonction wrapper capable de réagir aux statuts HTTP.
- Si l'API renvoie un 401, le script suspend temporairement le flux utilisateur et tente d'appeler la route de secours `/refresh`.
- Si le rafraîchissement réussit, la variable `response` est écrasée par une nouvelle tentative identique à la requête initiale. Pour l'utilisateur, c'est transparent.
- Si la route `/refresh` échoue à son tour (*refreshToken* expiré ou supprimé de la BDD), le script redirige vers `login.html`.



CONCLUSION

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

SYNTHÈSE DES BONNES PRATIQUES

1. L'intégrité face aux falsifications

Si un utilisateur modifie son rôle `"USER"` en `"ADMIN"` dans le payload, la signature cryptographique devient mathématiquement fausse. La méthode `jwt.verify()` lèvera une erreur et rejettera le jeton. Le **JWT** est lisible mais infalsifiable.

2. Ne jamais stocker de secrets dans le payload

N'y placez aucun mot de passe ou donnée sensible car le Base64Url est décodable par n'importe qui.

3. La protection des cookies

Utilisez toujours les flags `httpOnly` (anti-XSS) et `sameSite` (anti-CSRF).

4. Le chiffrement du transport

En production, passez le flag `secure: true` pour exiger le protocole **HTTPS**, sans quoi le jeton circule en clair sur le réseau.

SESSION VS JWT

Critère	Sessions (Express-Session)	JSON Web Token (JWT)
Lieu de stockage de l'état	Serveur (RAM ou BDD SQLite)	Client (dans le jeton lui-même)
Scalabilité (plusieurs serveurs)	Complexe (nécessite un store partagé comme Redis)	Native et facile (totalement stateless)
Révocation instantanée	Oui (on supprime la clé en RAM)	Non (il faut attendre l'expiration du jeton d'accès)
Poids sur le réseau	Très léger (juste un UUID de 36 caractères)	Plus lourd (en fonction de la taille du payload)
Cas d'usage idéal	Sites web classiques, e-commerce, etc.	APIs REST, Microservices, app mobiles, etc.

BILAN

Voyons le **protocole JWT** en **4 étapes** clés :

1. **L'authentification** (`POST /login`) : Le client envoie ses identifiants validés par le serveur avec Bcrypt. Le serveur génère un *accessToken* (éphémère) contenant le profil de l'utilisateur ET un *refreshToken* (clé opaque de 7 jours stockée en base). **Rien n'est stocké dans la RAM du serveur !**
2. **L'empreinte** (`Set-Cookie`) : Le serveur renvoie ces deux jetons au navigateur via deux cookies distincts (avec les directives de sécurité).
3. **La résilience** : Lors des requêtes suivantes, le navigateur joint automatiquement les cookies. Si l'*accessToken* expire, le frontend intercepte l'erreur 401, sollicite la route `POST /refresh` avec son *refreshToken* toujours valide et récupère un nouvel *accessToken* (transparent pour l'utilisateur).
4. **La révocation** : Le serveur ordonne au navigateur de détruire les deux cookies, supprime la ligne du *refreshToken* dans la base (invalidant tout rafraîchissement futur).

UNIVERSALITÉ DU JWT

Le JWT n'est pas une exclusivité de l'écosystème Node.js ! C'est un **standard RFC universel**.

Vous retrouverez exactement la même logique dans vos projets **Symfony** ou **PHP** natif, par exemple.

-  Exemple avec la librairie officielle **Firebase\JWT** en **PHP** :

```
Exemple PHP

use Firebase\JWT\JWT;
use Firebase\JWT\Key;

$secretKey = $_ENV['JWT_SECRET'];

// 1. Génération du jeton (équivalent de jwt.sign)
$payload = [
    'id' => 42,
    'email' => 'bruce@wayne.com',
    'exp' => time() + 900 // 15 minutes
];
$jwt = JWT::encode($payload, $secretKey, 'HS256');

// 2. Vérification du jeton (équivalent de jwt.verify)
try {
    $decoded = JWT::decode($jwt, new Key($secretKey, 'HS256'));
    $userEmail = $decoded->email;
} catch (Exception $e) {
    // Jeton falsifié ou expiré !
}
```

QUIZ ORAL

1. Quels sont les **trois composants** qui forment la structure visuelle d'un JWT ?
2. Est-il possible de lire le contenu du **payload** d'un JWT sans posséder la **clé secrète** du serveur ?
3. Quel risque majeur encourez-vous si vous stockez votre JWT dans le **localStorage** de votre frontend ?
4. Pourquoi est-il techniquement impossible de "détruire" instantanément un JWT côté serveur avant sa date d'expiration ?



COMPLÉMENTS

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

*1 - LOAD BALANCING

Le **Load Balancing** (ou *répartition de charge*) est une technique d'architecture réseau consistant à placer un serveur intermédiaire devant une flotte de plusieurs serveurs d'application.

Ce serveur intermédiaire est appelé « **Load Balancer** ».

Son rôle est de distribuer les requêtes HTTP entrantes des utilisateurs de manière équitable et intelligente entre ces différents serveurs.

Si un serveur est surchargé ou crashe, le **Load Balancer** redirige automatiquement le trafic vers les serveurs restants disponibles.

*2 - TYPE DE JETON

Le **type de jeton** est une métadonnée qui indique au serveur **la nature** du document qu'il est en train de lire.

Dans 99% des cas, cette valeur contient textuellement "JWT". Il s'agit d'une sécurité de premier niveau pour le serveur.

Ainsi, si un développeur distrait envoie un autre type de document signé à l'API (comme un jeton d'un autre protocole), le middleware lit le header, voit qu'il n'est pas écrit "JWT" et rejette la requête sans perdre de temps à faire des calculs mathématiques lourds.

*3 - ALGORITHME DE SIGNATURE

L'**algorithme de signature** est une **recette mathématique** utilisée pour **générer le sceau** de sécurité du jeton. Il indique au serveur la formule exacte qu'il doit appliquer pour vérifier si le jeton est authentique.

Il existe **deux grandes familles** d'algorithme de signature :

1. Les algorithmes symétriques (ex: HS256 / HMAC SHA256)

Ils utilisent **une seule et unique clé secrète** pour créer ET pour vérifier le jeton.

Dans le cas du JWT, il s'agit du **secret** stocké dans le fichier `.env` du serveur. C'est l'approche idéale quand le serveur qui crée le token est le même que celui qui le vérifie.

2. Les algorithmes asymétriques (ex: RS256 / RSA, ou ES256 / ECDSA)

Ils utilisent un couple de clés (comme pour SSH) :

- Une **clé privée** (secrète) détenue uniquement par le serveur d'authentification pour *générer* le token.
- Une **clé publique** (diffusable) donnée à tous les autres serveurs ou API pour qu'ils puissent *vérifier* la signature.

*4 - BASE64 VS BASE64URL

Le **header** et le **payload** d'un JWT ne sont pas en Base64 mais en **Base64Url**. Pourquoi cette nuance ?

Le problème du Base64 classique sur le Web

L'alphabet du Base64 standard utilise les lettres, les chiffres, mais se termine par trois caractères spéciaux : `+`, `/` et `=` (utilisé comme remplissage à la fin) mais ces caractères ont des significations réservées et strictes dans le protocole HTTP (le `/` sépare les routes et le `+` est souvent converti en espace par les serveurs, par exemple).

 Si votre JWT contient ces caractères, il risque d'être déformé ou coupé lors du transit dans un cookie ou une URL.

La solution : Le Base64Url

Le standard JWT utilise donc une variante "Web-Safe" (sécurisée pour les URLs) qui nettoie ces caractères : le `+` est remplacé par un tiret (`-`), le `/` par un underscore (`_`) et le `=` est **supprimé**.

Exemples :

- **Base64** => `eyJzdWIiOiIx...i6IkJydWNlIn0+L3c=`
- **Base64Url** => `eyJzdWIiOiIx...i6IkJydWNlIn0-X3c`

*5 - LOCALSTORAGE

Le **localStorage** est un espace de stockage persistant du navigateur web pour chaque site web via une API JavaScript très simple.

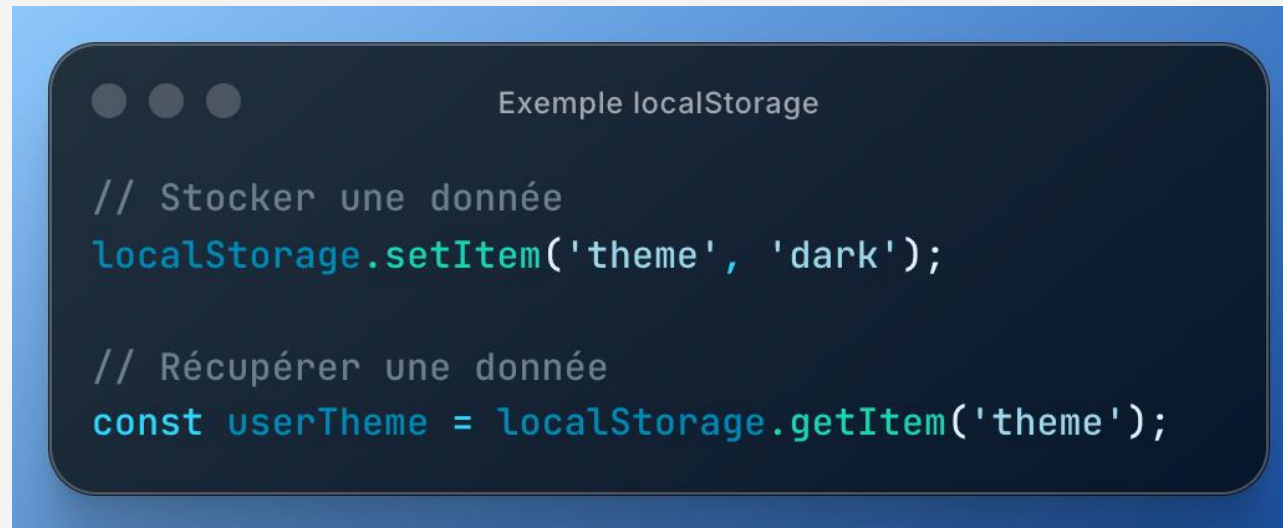
Il est capable de stocker environ 5Mo de données par domaine (beaucoup plus qu'un cookie limité à 4 Ko).

Ses données n'ont **pas de date d'expiration** et restent donc stockées sur le disque dur de l'utilisateur même s'il ferme l'onglet ou qu'il éteint son ordinateur.

Contrairement aux cookies, les données du localStorage **ne sont jamais envoyées automatiquement** au serveur lors des requêtes HTTP.

*5 – LOCALSTORAGE

Pour stocker ou lire une information, le développeur front doit écrire du code manuellement :



```
Exemple localStorage

// Stocker une donnée
localStorage.setItem('theme', 'dark');

// Récupérer une donnée
const userTheme = localStorage.getItem('theme');
```

`localStorage.getItem()` est donc accessible par **n'importe quelle ligne de code JavaScript** qui s'exécute sur un site web. Si un pirate réussit à injecter un script malveillant sur votre page (via une faille XSS, par exemple), il pourra faire `localStorage.getItem('jwt')` pour voler le jeton de l'utilisateur.

*6 - DOTENV

Pour signer numériquement nos JWT, on utilise une **clé secrète**. Un **secret** ne doit jamais être écrit en clair dans le code source !

Le module `dotenv` charge des **variables d'environnement** depuis un fichier masqué `.env` situé à la racine du projet et les injecte dans l'objet global Node.js `process.env`.

Le fichier `.env` va contenir donc des secrets d'infrastructure propres à votre machine. Il doit **impérativement** être ajouté au fichier `.gitignore` pour ne jamais être publié !



DES QUESTIONS ?

CHEVALIER@CHRIS-FREELANCE.COM